

REGULATION ROTATION — what has to change when a new Champions regulation goes live

Version: 1.0.0 — 2026-09-24. Line: abra/regmc — CHANGELOG-REGMC.md .

Opened 2026-09-20, during the Reg M-B → M-C rotation. Will: "identify all the things we need to update in between regulations and make a list so its easier for next time".

READ THIS FIRST: THIS PAGE IS NOT THE LIST

The list is **printed**, not typed. A typed inventory of every file that names a regulation is the fourteen stale handoffs in a new costume — and it would be a longer list than any of them:

```
node engine/regulation_touchpoints.js      # every tracked file naming a regulation, classified
node engine/regulation_touchpoints.js --rules  # how each class was decided, so you can check it
node engine/next_regulation.js --checklist    # the readers of the config, and the order of the flip
node engine/format_id_scan.js              # typed format-id call sites under engine/ build/ tests/
```

What this page carries is the part **no scan can derive**: the ORDER the steps unblock each other in, and the judgements a person has to make. Every step below is marked:

- **DERIVED** — a command prints the answer. Run it; do not remember it.
- **JUDGEMENT** — a person decides, and the decision is written down with its reason.

The classes `regulation_touchpoints.js` reports mean different work, and the counts are useless undifferentiated:

class	what the rotation does
CONFIGURATION	EDIT it. The id is a setting and everything downstream follows it.
HARDCODED	DECIDE, per site. After the flip it keeps working, about the OLD regulation, reporting success.
DERIVED	RE-RUN its generator. No edit — but the figure is about the old regulation until you do.
COMMENTED	Read once. The id is only in a comment: stale prose living inside live code.
PROSE	Rewrite at the documentation pass. Judgement, never a find-and-replace.
HISTORICAL	LEAVE IT. Dated evidence, frozen releases, archived stores, findings records. Rewriting these edits the evidence chain, which this repository forbids.

THE ORDER

1. Collect the new regulation from the day it exists — DERIVED, and already automatic

`engine/next_regulation.js` derives candidate format ids by SHAPE and probes three authorities; the hourly `.github/workflows/next-regulation.yml` collects them. Nothing here needs an edit for a regulation to start being stored.

Collectable and simulatable are two different questions, and the checkout LAGS. A format can be ladderred for days before the local Showdown checkout carries it. `next_regulation.js` reports the gap rather than smoothing it; believe the report.

2. Take a SECOND checkout. Never pull the one the closing regulation's figures rest on — JUDGEMENT

This is the lesson the M-C rotation paid for first. A single Showdown checkout that gets pulled moves the authority **under the published figures of the regulation that is closing** — the same failure as a measurement reading a live store, one level up. The two checkouts and their roles are recorded in `docs/REGMC.md`.

The outgoing checkout becomes **pinned**: never pull it, never build it, never modify it. Prove the separation disturbed nothing rather than assuming it — the M-C pass compared the outgoing format's legal species set across both checkouts and found it identical (CHANGELOG 7.1.0).

3. Derive the delta AGAINST THE PINNED AUTHORITY — DERIVED, and the shortcut is a trap

The new checkout ships a backward-compatibility mod for the old regulation, so it is tempting to derive both sides from one set of bytes. **That mod is not a faithful reproduction of what ABRA was built against.** On this rotation it failed to revert the moves the delta names, so the same-bytes walk reported *moves +0* where the honest answer against ABRA's pinned authority was a non-zero count (`docs/_reports/2026-09-20-regmc-delta.md` §1). Both numbers are true statements about different questions; only one of them describes the work.

Four things the delta must ask, because three of them are invisible to a legality list:

1. **Added and removed**, by id, for species, moves, items and abilities — the easy half.
2. **Entries legal in BOTH regulations whose DEFINITION moved.** On this rotation two moves had their PP halved (§3.1). Every *what was added / what was removed* list shows nothing, and an engine built as *the old regulation plus the new things* carries the old value forever.
3. **The ruleset and the ban list, asked of the format rather than recalled.** A ban that LIFTS reads as no change at all in an added/removed list. One did on this rotation (§6), and it is named in the umbrella `CLAUDE.md` ban list, which becomes wrong the day the new line goes active.
4. **Learnsets, confirmed with the `TeamValidator` in both directions.** The first instrument used on this rotation climbed the pre-evolution chain and reported zero changed species; the validator disagreed for a heavily-used one (§4). A control that returns opposite verdicts across the knob is what made the second answer believable.

Count MECHANICS, not species. A third of the added species on this rotation brought no new ability and no new move (§5). Counting species overstates the work and points it at the wrong things.

4. Re-admit what the strict legality filter drops — JUDGEMENT, with a printed list

`CLAUDE.md` requires every dex walk to be filtered, and that rule is right. On this rotation the filter **silently dropped a live ability** carrying `isNonstandard: "Future"` — the validator accepts a legal set carrying it and it has a real handler body (`docs/_reports/2026-09-20-regmc-delta.md` §7). A generator or a census walk built on the strict filter would never see it, never stage it, and report full coverage while doing so: **a capability absent with everything reporting success.**

Will, 2026-09-20: legality is what the `TeamValidator` accepts. A flag is evidence, never the answer. So: filter as before, then re-admit the flagged entries the validator accepts, and **print the re-admitted list on every run** so it can never be silently empty.

5. Decide the scope — JUDGEMENT

Which information regime (open sheets, bo1, bo3), which exclusions, which population is in scope. The M-C decision was *the same scope as the outgoing regulation, for now* — explicitly a decision to revisit once the format is understood, because added mechanics can change what is worth modelling (`docs/REGMC.md`). Nothing derives this. Write down the decision and the words it was made in.

6. Check whether the upstream RULES moved mid-regulation, and cut by the CONJUNCTION — JUDGEMENT

A regulation's rules are not fixed on its start date. On this rotation an upstream Showdown commit changed an item interaction **days after the format went live**, so replays collected in that window were played under a different rule regime than the ones after it. Any measurement over that store must exclude the window or account for it.

The exclusion is a CONJUNCTION, not a date range. Will: "*i mean can we just remove the games that have an eject button on the team sheet*". A game leaves the pool only if it was played before the fix **AND** the item appears on a team sheet; the rule change cannot have altered a game the item never appears in. The date cut alone was proposed, called cheap without being measured, and would have discarded roughly twenty-five times more evidence than the change could possibly have affected (CHANGELOG 7.2.0).

And the conjunction is exact rather than approximate, because of the information regime. Under open team sheets every item on the field is declared by one of the two players, so scanning BOTH sheets catches an item transferred mid-battle. That was measured, not assumed (CHANGELOG 7.2.0). Under closed sheets the same argument does not hold and the cut would have to be wider — **check the regime before reusing this reasoning.**

7. Archive the outgoing regulation — DERIVED, and it is the one unrecoverable step

```
node build/triggers.js           # says whether the archive is already due
node build/archive-regulation.js  # snapshots store + raw logs + models into data/archive/<reg>/
```

Do this **before** the flip. Everything it writes is HISTORICAL from that moment.

8. Flip the config — DERIVED list, JUDGEMENT on when

`data/regulations.json` is the single place the active format id is a setting. Setting `active` simultaneously re-points the ladder collector, re-labels the model CHOMP reads, and re-points the simulator at a format the pinned checkout may not carry.

```
node engine/next_regulation.js --checklist  # the block to paste, the direct readers, the order
```

AND `active` IS NO LONGER THE ONLY WAY TO SAY WHICH REGULATION A RUN IS ABOUT — 2026-09-21. It is the DEFAULT, and a run may now name one instead:

```
node <anything> --regulation regmc          # or ABRA_REGULATION=regmc; both bring the right checkout
```

`engine/regulation.js` is the single resolver and `champions_sim.FORMAT` reads it, so every caller follows with no edit. **This matters to the ORDER of this page:** flipping `active` used to be the only way to run the incoming regulation at all, so it had to happen early and it moved the whole repository at once. It can now happen when the outgoing regulation is genuinely finished, with both runnable side by side in the meantime. A run that rewrites shared config is a run that can corrupt another one beside it, and two agents on two regulations share one `active` key.

Two things it does NOT change, and both are the point of the two maps:

- `data/regulations.json` **has a second map, runtime, and the incoming regulation goes THERE first.** `engine/next_regulation.js` walks `regulations` to decide what is already known, so adding the incoming id to `regulations` reclassifies it from `candidate` to `known` and the hourly collector quietly stops collecting it. **Being selectable is not being active.**
- Everything a MEASUREMENT pins is still separate** — step 9 below is unaffected. The flag selects a format and a checkout; it does not select a pool, a census or a release.

Three things the config does NOT control, and each one is a separate decision:

- Which Showdown checkout is the authority.** ~~That is resolved by `engine/showdown_path.js` from a sibling directory, not from the config. A second checkout (step 2) therefore needs an explicit `SHOWDOWN_PATH`, per run or per shell.~~ **Superseded 2026-09-21:** the checkout is now part of the regulation. `runtime.<id>.checkout` in `data/regulations.json` names it and `engine/showdown_path.js` tries it first, so selecting a regulation selects its authority. An explicit `SHOWDOWN_PATH` still wins over all of it, for a checkout kept somewhere else. **The pinned commit moved with it** — `champions_sim.PINNED_COMMIT` was a literal and one constant cannot pin two authorities.
- The silent fallbacks.** Several scripts read the config inside a `try` and fall back to a hardcoded format id when the read fails — deliberately, so a batch job guesses rather than crashes. `engine/champions_sim.js` ANNOUNCES its fallback on stderr and records why; the others are silent. Run `node engine/regulation_touchpoints.js --class hardcoded` for the live list. **A silent fallback after a flip is this project's signature failure mode:** nothing is broken, nothing reports a failure, and every downstream figure is about the previous regulation.
- The pinned corpora** — step 9.

9. Re-point what a MEASUREMENT pins — three things, none of them the config

A frozen release pins the engine SOURCE files. It does not pin the store, the pool, the census or the artifacts, and a rotation moves all four. `regulation_touchpoints.js` prints the frozen corpora it can find and how many live source lines name each — **it cannot see them by id, because a pinned path carries no regulation id at all.**

pin	what the rotation does
the frozen team pool (<code>--team-store</code>)	The new regulation gets its own pinned pool, cut from its own store with step 6's exclusion applied. The old one is the old regulation by construction and is never re-pointed.
the census pin	Regenerate for the new format. A count taken either side of a census regeneration is not a before/after .

pin	what the rotation does
the gate's lattices (<code>--games values</code>)	<code>engine/quarantine.js</code> requires zero on several sample sizes chosen by walking the swarm builder so the samples share few teams. A zero that holds at one sample size is a fact about that sample. Re-derive the sizes against the new pool with <code>node engine/lattice_walk.js --regulation <new> --anchor 1200 --from 1250 --to 2000 --greedy 2</code> and add one row to <code>LATTICE_GAMES</code> in <code>engine/quarantine.js</code> .
every gate artifact	Nothing to re-point: <code>engine/regulation.js artifactFor</code> sends every artifact the gate reads or an instrument writes to <code>data/<name>--<new>.<ext></code> (pool: <code>data/team-pool-frozen-<new></code>) for any regulation but Reg M-B, and refuses a write onto any other existing <code>data/</code> file. Run the gate with <code>--regulation <new></code> . A new artifact family gets a pattern in <code>PER_REGULATION_ARTIFACTS</code> ; <code>tests/test-regulation-artifacts.js</code> fails until it has one.
the release set	Every release cut before the flip describes the previous regulation. Cut the first new one only after the rebuild and re-point steps below have settled, or it freezes the old format.

10. Rebuild what the format decides — DERIVED

Every artifact in the DERIVED class is regenerated, not edited. `--checklist` prints the builders in dependency order. Each is a frozen-release source or an input to one, so a release cut before they settle freezes the previous regulation.

11. Decide every HARDCODED site — DERIVED list, JUDGEMENT per site

```
node engine/regulation_touchpoints.js --class hardcoded
```

This is the list worth shrinking, and it is NOT a find-and-replace. Three kinds live in it and they want opposite treatment:

(Shrunk on 2026-09-21 — the counts are in `CHANGELOG-REGMC.md 0.10.0` and the live list is PRINTED by the command above, never typed here. Every site closed was the FIRST kind below: an inlined `JSON.parse(readFileSync('data/regulations.json'))` in a `try`, with its own copy of the format literal in the `catch`, and all but one of them silent when it fell back. They read `engine/regulation.js` now, which carries the one surviving literal and answers the DEFAULT path only. What remains is the second and third kinds — workflows, UI pages, the M-C pool cutter and probe fixtures pinned to the format they were measured on, including a deliberate Reg M-A control. Full account: `docs/_reports/2026-09-21-regulation-runtime.md §7`.)

- **A fallback or a default in engine code** — should read the config, and should say so out loud when it cannot. This is the avoidable cost.
- **A pin on what was MEASURED** — a replay id, a probe fixture, a deliberate old-regulation control. **Rewriting one edits the record.** `engine/format_id_scan.js` states this doctrine and takes no position on which sites are wrong, correctly: the caller decides.
- **A workflow or a UI string** — a shell loop naming the format, a placeholder in a page. Cheap, and visible to a user, so it is the kind that survives longest unnoticed.

12. Re-measure. Nothing carries over — JUDGEMENT

A figure measured under the old regulation is not stale, it answers a question nobody is asking any more. Under this repository's versioning that is a **basis change**, so it is a MAJOR and the old figures come OUT of the living documents rather than being captioned. A caption is not a quarantine.

```
node engine/status.js      # what is stale, what is withheld, what is owed
node engine/provenance.js
node engine/open_work.js
```

13. The documents — JUDGEMENT

The ban list in the umbrella `CLAUDE.md` is a *mechanism, not a list* and must be re-asked of the new format. The white paper, the deck, the technical documentation, `docs/SUMMARY.md` and `docs/MODELS.md` are rewritten at the major, from the running-notes rows. `regulation_touchpoints.js --class prose` prints which documents still name the old regulation.

14. Downstream consumers — CHOMP, and what ABRA itself reads FROM CHOMP — JUDGEMENT, deferrable

CHOMP is a separate project (`../CHOMP`) and a CONSUMER. It never learns about a rotation on its own. Two directions, and both are checked every rotation:

- **What CHOMP reads from ABRA.** CHOMP loads the usage model (`data/meta-usage.json`, and for a new regulation its sibling, e.g. `data/meta-usage-regmc.json`), keyed by CHOMP's OWN species table, which is built from the OLD regulation. A species missing from that table scores as zero usage and nothing warns. The three changes CHOMP needs, file by file, are in `docs/_reports/2026-09-22-regmc-usage.md`. **Deferring this is allowed** (Will, 2026-09-22: "*chomp update will come later*"). While it is deferred, do not re-point `data/meta-usage.json`: CHOMP keeps reading the old regulation's file, which is at least internally consistent.
- **What ABRA reads from CHOMP.** Some ABRA builders and tests still read CHOMP files, so a stale CHOMP silently feeds the old regulation back in. Print the live list, never a typed one:

```
`` bash grep -rn "CHOMP" engine build tests --include=*.js --include=*.py | grep -v graveyard | grep "path.join\\|require" ``
```

The M-B species table builder `build/build_engine_data.js` reads CHOMP's model. That is why the new regulation got its own builder (`build/build_engine_data_regmc.js`) that derives from the format instead.

(Changed 2026-09-24, in the Reg M-C 1.0.0 documentation pass; the text above is left as written for this rotation.) **From Reg M-C on, CHOMP is not a downstream consumer.** Will moved it inside ABRA as the team-preview solver (`solver/chomp/`, `solver/PLAN.md §2`), and the `../CHOMP` repository is reference only. So on the NEXT rotation this step shrinks to the second bullet — find what ABRA still reads from `../CHOMP` and cut it — and CHOMP rotates with the rest of the solver in step 15.

15. The models downstream of the engine — JUDGEMENT, and on Reg M-C the answer was rebuild

Every model that plays on MEDICHAM or was fitted on its features answers a question about the OLD regulation. On the M-B → M-C rotation Will's call (2026-09-24) was to **retire the Reg M-B models rather than port them** and rebuild every one except MEDICHAM from scratch under `solver/`, for open sheets only. What that decision needs, whichever way it goes next time:

- **Store-only models can start the day the store exists.** The human dataset, MAG, DODUO, XATU's bring model and GURU never touch the simulator, so they do not wait for the gate.
- **Anything that plays games waits for the new gate,** and whatever it measured before the gate opened is PRE-GATE: withheld until re-run on the release the gate opened on, never captioned.
- **The old model files are archived in the commit where each replacement passes its exit test,** not when the rotation starts (`solver/PLAN.md §8`). Several sit in `engine/engine_release.js SOURCES`; run `node engine/engine_release.js compat` before moving one.

WHAT CANNOT BE DERIVED, AND MUST BE JUDGED EVERY TIME

A runbook that pretends judgement is mechanical is worse than none. These do not have commands:

- 1. **Scope.** Which information regime is in scope, which exclusions stand, whether the new mechanics change the answer. A derivation can list what arrived; it cannot say what is worth modelling.
- 2. **What to exclude, and on what predicate.** The date-versus-conjunction choice on this rotation was the difference between discarding a third of a store and discarding barely any of it — and the conjunction is only exact because of the information regime. Measure the cut before taking it.
- 3. **Whether a figure is comparable across regulations.** Usually it is not: the basis moved. But "usually" is not a rule, and `engine/arms_comparable.js` answers only for one artifact and only within one regulation. State the claim; do not let a number drift across the boundary unlabelled.
- 4. **Which HARDCODED sites are defects and which are pins.** Structural scanning finds the sites. Only a reader can tell a stale default from a deliberate record of what was measured.
- 5. **When to flip.** Collectable, simulatable and *worth switching to* are three different dates.
- 6. **What a divergence MEANS in a format nobody has played yet.** The new mechanics have no baseline, so the first runs have nothing to be a regression against.
- 7. **Which CUSTOM-RULE rooms are contamination and which are the target population** — added 0.6.0, and it is a genuinely different answer per regulation. A tournament room played under Showdown's custom rules is indexed under the BASE format id, so `search.json?format=` pulls it into the ordinary store; `engine/durable-ingest.js` filters on the format id and a format id does not encode custom rules. On the Reg M-B ladder that is a twentieth of the store and all of it is out of scope (`data/quality-filter.json` `exclude_custom_ruleset`). **On Reg M-C the same scan gives the opposite answer:** nearly every custom-rule game in `data/team-pool-frozen-regmc/games.ots.jsonl` is a `Force Open Team Sheets` room — and that rule is what makes a ladder game open-sheet, which is the M-C scope. Copying the M-B rule across would delete the evidence the scope was chosen to collect. The counts are in `docs/REGMC.md` and the full account in `docs/_reports/2026-09-21-custom-ruleset-filter.md` . **DERIVED** for the counts, **JUDGEMENT** for the cut:

```
`` bash node engine/scan_custom_rulesets.js \ --raw data/games.<store>.raw-logs.jsonl --store data/games.<store>.jsonl --out /tmp/scan.json ``
```

Read the UNTESTABLE line before reading anything else: the infobox is in the RAW log, so the answer is always a floor. On Reg M-C that line says the great majority of the store has no local raw log — which does not mean *M-C is nearly clean*, it means *M-C has barely been asked*.

FOUND DURING THE REBUILD — APPEND A ROW EVERY TIME ONE BITES

(Will, 2026-09-21: "make sure to continuously update the regulation change document so next regulation change is even easier".) **This section is appended in the same commit as the fix that found it.** One row per trap: what bit, how it showed, what to do next time. The steps above say what to do; this says what went wrong while doing it, in the order it happened on Reg M-B → M-C.

what bit	how it showed	next time
Three files are per-regulation, not one. The species table, <code>data/tags.json</code> and <code>data/protocol-events.json</code> are each derived from the format.	The first new-regulation games parted on things the new format added (terrain setters, new mega stones) because the OLD tag file had no row for them.	Add the three to <code>runtime.<reg></code> in <code>data/regulations.json</code> on day one; <code>fileFor</code> in <code>engine/regulation.js</code> redirects all three and refuses a write onto the old regulation's files. Derive each with the new checkout.
A reader that loads a tag file by path bypasses the redirect. <code>engine/names.js</code> read Reg M-B's tags under Reg M-C.	The differential picked Reg M-C teams by Reg M-B membership. Nothing failed.	After adding the per-regulation files, grep for every direct load of <code>tags.json</code> / <code>protocol-events.json</code> / <code>engine-data.js</code> and route it through the selector.
Editing a DERIVER restales the old regulation's artifact. 0.14.0 added <code>--out</code> to <code>engine/derive_protocol_events.js</code> without regenerating Reg M-B's <code>data/protocol-events.json</code> .	Every differential, roster and battery run refused to start with "produced by ... and that file is now ...". A battery script that greps an OLD artifact afterwards then printed last run's clean numbers as if they were new.	Any edit to a deriver: regenerate the old regulation's artifact in the same commit and show it is byte-identical apart from its stamps. Read each run's exit code before any figure a wrapper prints.
The pre-commit audit copied the whole Projects folder into <code>%TEMP%. engine/artifact_audit.js</code> read <code>path.join(ROOT, '..', '..', '..', '..', '..', c)</code> in <code>engine/regulation.js</code> as a sibling checkout named <code>..</code> .	Every commit hung silently for minutes; stray copies (<code>%TEMP%/Pokemon</code> , <code>agents</code> , <code>worktrees</code>) piled up.	Fixed 0.15.0 (a name made of dots is not a sibling). If a commit hook goes silent, look at <code>%TEMP%</code> for a fresh copy of the repo's parent before anything else.
The new regulation's smoke IS the work list.	The first Reg M-C games played end to end the first time, and most of those that parted did so on terrain set on entry (<code>docs/_reports/2026-09-21-regmc-table-wiring.md</code>).	Build the species table, run a small unpinned smoke immediately, and rank by FIRST CAUSE. It found the headline features of the new format before anyone read a patch note.
New mega stones arrive untagged.	"Mega forme did not evolve" in the first smoke (<code>docs/_reports/2026-09-21-regmc-tags.md</code>).	Check every legal stone in the new format has its mega tag right after deriving the tag file.
The closed regulation needs a regression check, not a re-measure.	A full Reg M-B battery was re-run on 2026-09-21 and was not needed.	For the closed line: damage differential identical plus one lattice at 0. Anything more is re-measuring a published result.
A whole item class came back from Past with no mechanic tag. The terrain seeds were illegal in Reg M-B, so the old tag deriver never needed a rule for them; in the new tag file they carried only <code>flingable</code> , and tag_dex's coverage report did not flag them because <code>flingable</code> counts as a tag.	The largest first cause of the smoke by far, as a missing <code>-enditem</code> (<code>docs/_reports/2026-09-21-regmc-tags.md</code> §5, <code>docs/_reports/2026-09-21-regmc-seeds.md</code> §5).	After deriving the new tag file, list every item, ability and move that is legal now and was Past before, and read its handlers. Treat "only generic tags" (<code>flingable</code> , <code>pp</code> , <code>targetClass</code> , ...) as UNTAGGED. Rocky Helmet, Air Balloon and the Eject items are the same class.
A first-cause label can name the wrong mechanism. The smoke card read "Grassy Terrain heal ORDER"; the authority was skipping a semi-invulnerable body.	One game, two heal lines in a different order (<code>docs/_reports/2026-09-21-regmc-seeds.md</code> §0).	Open the card's before lines before naming a cause. And ask whether the real mechanism also exists in the closed regulation: this one did, so the fix moved the closed line's engine and needed the row-7 regression check. A census row for it would move a figure the closed line published, so it

what bit	how it showed	next time
		was withheld for a decision (see the next row).
The census belongs to the closed regulation. <code>data/mechanics-census.json</code> is built from Reg M-B's table and tags, so a mechanic legal only in the new format has no row it can go live in, and a row for a shared mechanic moves the census count the closed line published.	The seeds have a staged probe and no census row; the heal row went live and was withdrawn when <code>tests/test-docs-current.js</code> refused the moved count in three published documents (<code>docs/_reports/2026-09-21-regmc-seeds.md</code> §0).	Give the new regulation its own census file early (the same <code>runtime.<reg></code> map as the per-regulation files above). Until then, prove a mechanic with a <code>--regulation</code> probe and a knob, and say so in the changelog.
The gate and every instrument wrote ONE fixed path. Only the three engine files were per-regulation.	A Reg M-C <code>--write</code> would have overwritten Reg M-B's published evidence and exited 0, and the gate could only answer for Reg M-B (<code>docs/_reports/2026-09-21-regmc-gate.md</code>).	Fixed 0.16.0: per-regulation artifacts are <code>data/<name>-<reg>.<ext></code> ; see step 9's pin table. Run the gate with <code>--regulation <reg></code> . The lattice sizes do NOT transfer: re-derive them with <code>engine/lattice_walk.js</code> .
An absent input passed a clause. The open-defect clause read no register verdicts for the new regulation and passed.	The first Reg M-C gate run showed PASS with 0 verdicts read.	Fixed 0.16.0: absent verdicts read CANNOT-ANSWER. Run the new regulation's gate once before its instruments exist. Every clause must read NO ARTIFACT or CANNOT-ANSWER. A PASS there is a clause that cannot see absence.
A test that writes the old regulation's names into a sandbox becomes their writer. <code>engine/provenance.js</code> credits whoever names an artifact beside a write into <code>data/</code> .	The gate's artifact closure moved under a new test; the report records it (<code>docs/_reports/2026-09-21-regmc-gate.md</code> section 2).	Found 0.16.0. In a test, assemble artifact names, never spell them next to a write. Re-derive <code>node engine/provenance.js --graph --json</code> and check nothing credits the test.
Some builders read the old regulation's store whatever is selected. <code>engine/click_counts.js</code> and <code>engine/sheet_usage.js</code> read the Reg M-B stores by name, and the steering inputs are Reg M-B's too.	Under the new regulation they would write old-store numbers into correctly named new files. Nothing would fail.	Found 0.16.0, not fixed. Before running a builder for the new regulation, check which store it opens. The not-yet-per-regulation inputs are listed in <code>tests/test-regulation-artifacts.js</code> (NOT_YET).
The steered path reads more than the builders the brief named. The empirical arm CLICKS out of the behaviour table (<code>data/move-priors.json</code> , read out of the release) and PRICES a switch off <code>data/rollout-switch-census.json</code> . Both were the old regulation's.	Found by reading <code>engine/game_differential.js</code> 's empirical block, not from the brief (<code>docs/_reports/2026-09-21-regmc-census.md</code> §2).	Fixed 0.19.0: <code>engine/regulation_stores.js</code> gives every builder the new regulation's frozen pool, and the behaviour table is a per-regulation engine file. Next time, list every <code>readFileSync</code> and <code>REL.read</code> in the steered path before building anything.
The artifact seam also redirects a release cut. <code>data/rollout-switch-census.json</code> is a release SOURCE. Declaring it per-regulation made the new regulation's cut read it through the seam.	The Reg M-C release froze the new census's bytes under the old file name, so the manifest pairs one name with the other file's digest (<code>docs/_reports/2026-09-21-regmc-census.md</code> §7).	Found 0.19.0, not fixed. Before declaring a file per-regulation, check <code>SOURCES</code> in <code>engine/engine_release.js</code> . A SOURCE belongs in the <code>fileFor</code> map (like the behaviour table) so the cut freezes it by its own name.
A builder that never loads <code>engine/regulation.js</code> has no seam. <code>engine/click_counts.js</code> did not.	Run with the new regulation selected, it would have overwritten the old regulation's file. The seam is installed only by a module that loads it.	Fixed 0.19.0 for the four steering builders. Every builder that writes into <code>data/</code> must load <code>engine/regulation.js</code> before it writes.
A positional-argument parser reads <code>--regulation</code> 's value as an operand. <code>engine/policy.js</code> took <code>--regulation regmc</code> as a store path.	Found while running it (<code>docs/_reports/2026-09-21-regmc-census.md</code> §7).	Fixed 0.19.0. When a script parses positionals by hand, exclude the value that follows <code>--regulation</code> , or select the regulation with <code>ABRA_REGULATION</code> .
A file written through the artifact seam has no discoverable writer. No literal in the code names a <code>-<id></code> path, so <code>engine/provenance.js</code> could not attribute the new regulation's behaviour table, its observed copy or its switch census.	The OLD regulation's gate listed them under "NO DISCOVERABLE WRITER", so Reg M-B's gate output moved because of Reg M-C files (<code>docs/_reports/2026-09-21-regmc-census.md</code> §5a).	Fixed 0.19.0: a builder writing through the seam stamps <code>by</code> into the artifact. Next time, run the old regulation's gate before and after committing new-regulation artifacts, and diff the outputs.
Every new-regulation file moves the old regulation's gate inventory count. <code>engine/quarantine.js</code> counts every file in <code>data/</code> .	Reg M-B's gate differed from HEAD in exactly one line, the inventory size, after the new census and steering inputs were committed.	Found 0.19.0, not fixed. The fix is a regulation-aware inventory. Because that fix changes what the old gate prints, it needs a decision.
A store test in a worktree can pass by comparing nothing. The worktree has no old-regulation stores, so "selects the literal stores" compared two empty lists.	It passed on the first run and could not have failed.	Fixed 0.19.0: <code>tests/test-regulation-steering.js</code> has a control arm with the old stores present in its sandbox. A selection test needs the thing selected to exist.
An item legal only in the new regulation is an untested surface, whatever the old line's gate says. The closed line's damage differential, lattice, census and roster were all built from its own legal set, so none of them ever staged Rocky Helmet, Air Balloon, Red Card or Eject Button, and a gate that was open for the old regulation says nothing about them.	Four of the first five causes of the new smoke were items that came back from Past (<code>docs/_reports/2026-09-21-regmc-seeds.md</code> §5, <code>docs/_reports/2026-09-21-regmc-items.md</code>). The engine had no tag, no consumer and no probe for any of them, and nothing failed.	Diff the legal item, ability and move sets of the two formats on day one. Every entity that is new gets a derived tag, a consumer, a knob and a <code>--regulation</code> probe before its smoke count is trusted. Rank them by the new smoke's first cause, not by the old census.
Fixing a cause can unmask a game the smoke had written off as VOID. The played count is fixed by the pool, but a VOID game (the two dice streams stopped agreeing) is excluded from board-material and comes back once the desync is gone.	After Rocky Helmet the smoke's VOID count fell by one and a new first cause (a damage value) appeared on that game (<code>docs/_reports/2026-09-21-regmc-items.md</code> §2).	Read the VOID count beside board-material after every fix, and open any first cause that appears on a game that was VOID before. It is a newly visible defect, not one the fix introduced, until its card says otherwise.
Making the measurement per-regulation stops every instrument that reads the old regulation's census. Once reads	The 0.16.0 seeds probe and the smoke both died on <code>steering: cannot</code>	Build the new regulation's census early. Until then pass <code>--census <file></code> to every

what bit	how it showed	next time
of <code>data/mechanics-census.json</code> go to the new regulation's sibling, the whole-game differential and every staged probe that loads it refuse at <code>require</code> , because the steering needs a census and the new regulation has none yet.	read the selection input ... <code>mechanics-census-regmc.json</code> the moment the per-regulation seam merged (<code>docs/_reports/2026-09-21-regmc-items.md</code> §0). Loud, which is right.	smoke, and give a SCRIPTED probe a declared one-row stub (<code>tests/regmc_probe_kit.js</code> <code>scriptedCensusPin</code>): its games never consult the census, and saying so is better than borrowing the old regulation's.
A smoke card can belong to the games played before it, not to the game it names. One game (a Wave Crash into an Aura Guard body) parted after the Rocky Helmet fix and not before, yet no helmet was in it.	Played alone with <code>--config</code> , it agreed on the same engine; after the next fix it vanished from the full run again (<code>docs/_reports/2026-09-21-regmc-items.md</code> §2).	Before blaming a fix for a new card, re-run that card's config alone on the same bytes. If it agrees alone, the cause is state carried across games in one process: record it as that, not as the fix's regression.
One item can carry a narration half and a board half, and only a knob per half says which. Air Balloon's entry announcement writes no board leaf; its pop does.	Under <code>MEDI_AIR_BALLOON_SILENT</code> the probe's protocol checks go red and its boards stay identical; under <code>MEDI_AIR_BALLOON_UNPOPPED</code> the boards part (<code>docs/_reports/2026-09-21-regmc-items.md</code> §2).	Give each half of a new item its own knob, and read the probe's board assertions separately from its line assertions before ranking the cause against the board-material bar.
A rule the authority changed mid-regulation belongs in the tag, derived from the handler the format resolves. Showdown changed Eject Button four days into Reg M-C.	The same derivation reads <code>cancelsSourceSwitch: false</code> under the new checkout and <code>true</code> under the old one (<code>docs/_reports/2026-09-21-regmc-items.md</code> §3). A typed rule would have been silently wrong on one of them.	Derive every rule parameter from the handler, so that moving the checkout moves the tag and the diff of the tag file shows the change. Keep a knob that restores the old rule and prove the probe goes red under it.
A staged probe cannot see a switch the engine does not make. The harness answers the authority's switch request by mirroring this engine's slot, so when the engine does not switch the request cannot be answered and the game stops there.	Under <code>MEDI_EJECT_BUTTON_INERT</code> the authority's own switch line disappears too, so the fixture check reads red for a reason that is the engine's (<code>docs/_reports/2026-09-21-regmc-items.md</code> §3).	For a mechanic that ends in a switch request, assert the authority's item line (the <code>-enditem</code>) as the fixture, and treat the switch line as an engine-and-authority check. Say which is which in the probe.
A version number taken in parallel is a merge conflict in five documents. ENGINE and MEASURE worked the new line at once; MEASURE landed <code>0.19.0</code> while ENGINE's commits already carried <code>0.19.0</code> and <code>0.20.0</code> .	The rebase stopped on the changelog, the notes page and the two version headers of every replayed commit (<code>docs/_reports/2026-09-21-regmc-items.md</code> §5).	When two divisions work one version line at once, have the coordinator hand out version numbers up front, or have each division keep its commits local until it merges and renumber in one rebase. Never publish a number twice.
An unpinned smoke and the pinned differential rank different things. The unpinned smoke's small sample held almost no Eject Button or Emergency Exit games; the pinned lattice held many of both.	The item order the smoke gave (<code>docs/_reports/2026-09-21-regmc-seeds.md</code> §5) was right at the top and badly wrong below it (<code>docs/_reports/2026-09-21-regmc-census.md</code> §6).	Use the smoke to find what exists; rank and measure on the pinned differential the gate will read, once the new regulation has its census.
The move-effects table is built for one regulation. <code>data/move-effects.js</code> is not in the per-regulation map, so a move new to the format has no row: no secondaries, no self effects, and one reader that assumed a row threw and lost the game.	Two pinned games threw on a null read, and a scan found legal Reg M-C moves with no row, several of them with secondaries (<code>docs/_reports/2026-09-21-regmc-items.md</code> §8).	Add every generated engine table to the per-regulation map on day one, not only the species table, the tags and the protocol events; then scan the new format's legal moves for a missing row in each. Fixed 0.24.0 for the move-effects table (<code>docs/_reports/2026-09-22-regmc-engine.md</code> §1).
A generator that writes several tables writes all of them for the selected regulation, and the write guard protects only MAPPED files. <code>build/build_browser_data.js</code> writes <code>data/move-effects.js</code> and <code>data/mega-formes.js</code> ; once the first was per-regulation, a Reg M-C run would have written the second, unmapped, straight over Reg M-B's.	Found while making the move-effects table per-regulation (<code>docs/_reports/2026-09-22-regmc-engine.md</code> §1). Nothing would have refused it.	When one output of a multi-output generator becomes per-regulation, route every output through <code>fileFor</code> and <code>SKIP</code> , loudly, any output the new regulation has no file for. Fixed 0.24.0.
Editing a file with a text-mode tool can silently change its line endings, and a release id is computed from the working-tree BYTES. Files unpinned in <code>.gitattributes</code> check out CRLF; a Python text-mode read and write turned them LF.	<code>tests/test-engine-release.js</code> failed its line-ending invariant on <code>engine/medicham2-browser.js</code> , and the release cut from the LF bytes had a different id from the one a checkout would cut (<code>docs/_reports/2026-09-22-regmc-engine.md</code> §1).	Edit in binary mode or preserve the file's own line ending. Check <code>git ls-files --eol</code> on every edited file before cutting a release.
A card that parts only after earlier games can be the DRIVER's state, not the engine's. The differential's driver keeps state across games beyond its coverage counters: <code>MEGA_PREFER_B</code> alternates which slot megas and is not in <code>driverSnap</code> , so replaying a game alone, or with the counters restored, can give a different mega choice.	The Reg M-C Aura Guard card "agreed alone and parted in the full run"; a fresh-process replay with the counters restored reproduced one game exactly and parted another at the mega line (<code>docs/_reports/2026-09-22-regmc-engine.md</code> §2). The real defect was an engine gap that only shows when the new mega forme is chosen.	Before calling a card state leakage, replay it in a FRESH process with the driver's state restored and diff the two traces line by line. The first differing line says whose state it is: a click or a mega choice is the driver's; a number after identical inputs is the engine's.
Regenerating the new regulation's tag file moves every usage-weighted block, not just the new rule. <code>engine/tag_dex.js --regulation <id></code> weights by the stores on disk, and a worktree's stores are not the ones the committed file was weighted by.	The first regeneration for Glaive Rush changed the usage-ordered <code>linkage</code> blocks and <code>sheet_entries</code> with no rule change at all	Regenerate, then diff structurally with usage and stamps ignored. Commit only the rows and descriptors the new rule changed, spliced onto the committed file, and keep its line endings.

what bit	how it showed	next time
	(docs/_reports/2026-09-22-regmc-engine.md §4).	
A volatile's _vol entry is not always a clock. The engine stored a duration for a member that declares one and a bare 1 for one that does not, then decremented both. A new regulation added a member of an existing family (per-turn stat drops) with no duration and a different end rule.	The new member ended on its first residual in this engine while the authority kept dropping stats every turn (docs/_reports/2026-09-22-regmc-engine.md §5). Nothing failed.	When a new regulation adds a member to an existing tag family, print the new member's params next to the old ones and check every field the consumer assumes is present (a duration, an end hook). Derive the missing rule into the tag rather than special-casing the name.
A move field can mean something other than its name. Revival Blessing carries selfSwitch: true so the simulator raises a switch request naming a FAINTED body; its user never leaves. Every rule that reads selfSwitch as "a pivot" played it as Parting Shot.	The largest board-material family on the pinned differential once the items were done, and the refused choices the harness logged ("You have to pass to a fainted Pokémon") (docs/_reports/2026-09-22-regmc-engine.md §6).	For every move new to a regulation, read its whole block, comments included, before trusting a field-derived tag. A comment in the authority that says a field is a workaround is a derivation input.
An item that comes back from Past can already carry the right tag and still do nothing. Terrain Extender had extendsDuration naming the four terrains in the new tag file, but the two terrain writers wrote a literal duration, because the only extender items the old regulation had were for weather and screens.	The terrain clock parted 5/8 in more pinned games than any other cause once the pivot defect was fixed (docs/_reports/2026-09-22-regmc-engine.md §7).	For every item new to a regulation, grep the engine for each of its tags' consumers, not just for the tag's existence. A tag with a consumer for only some of the effects it names is the same gap as no tag.
The closed regulation's census rows assert that regulation's BUILDS, not just its mechanics. Run on the new table, a row that types a Speed, a damage number, a moveset or an authority line reads MISSING while the mechanic works.	All the rows missing under the new regulation's census were staging gaps of that kind: typed stats and damage, fixture choices the new behaviour table no longer makes, a key spelling, and one authority line the new checkout dropped (docs/_reports/2026-09-22-regmc-engine.md §8).	Before treating a MISSING row in the new census as an engine gap, open its pass condition and look for a typed value. Derive expected values from the build and the authority on the run, as the new regulation's staged probes do.
A new door at the end of an action inherits every "after the move" hook that was placed on the post-action pass. Eject Button, Red Card and Emergency Exit made this engine switch bodies inside the action; White Herb's onAnyAfterMove had been served by the pass that runs after the action, which was harmless while no switch happened in between.	A switch-in Intimidate's drop was cleared by a herb the authority had already spent (docs/_reports/2026-09-22-regmc-engine.md §11).	When a new regulation adds a mechanic that acts at the tail of an action, list the hooks the authority raises between the move and that tail (AfterMove, AfterMoveSecondary, Update) and check where this engine serves each one.
The usage file CHOMP reads is per-regulation, and it was not. engine/analyze.js took its format token from active in data/regulations.json , not from the selection, read Reg M-B's ladder store by default, and wrote a fixed data/meta-usage.json . The test's NOT YET list excused it on the claim that analyze.js "has its own regulation handling", which it did not.	Under the new regulation it would have modelled the old store (or been refused by the artifact seam), and the differential's severity ranking, which reads the usage file LIVE, ranked the new regulation's bodies by the old one's usage. Nothing failed (docs/_reports/2026-09-22-regmc-usage.md §7).	Fixed abra/regmc 0.23.0: meta-usage.json is declared per regulation and analyze.js --regulation <id> writes data/meta-usage-<id>.json from that regulation's own stores. Next time, check every NOT YET reason against the code it names before trusting it.
A quality verdict computed over the old store removes nothing from the new one, and that looks clean. The legality verdict and the custom-ruleset scan are id lists built from the old regulation's store only.	Applied to the new regulation's games they cannot match an id, so their funnel steps read as zero removals, which is indistinguishable from a clean corpus (docs/_reports/2026-09-22-regmc-usage.md §2).	The usage model publishes them as NOT ASKED and runs its own legality audit instead. Next time, give every id-keyed verdict a regulation stamp, or refuse it for a regulation it never judged.
Closed-sheet games carry illegal species that no sheet can validate. The new b01 ladder holds a handful of rooms whose team preview shows species the format does not contain.	The strict filter caught them, but no stored sheet carried any of them, so the validator had nothing to judge and the first audit left them unresolved (docs/_reports/2026-09-22-regmc-usage.md §4).	Construct the fixture: the species alone, the item its forme requires, one move; set aside the team-size and o-Stat-Point lines. Audit the corpus beneath the table, not only the table, and report where each failure sits.
A consumer can read the new file and still be blind to the new meta. CHOMP keys usage by its own species table, vendored from the old regulation's data/engine-data.js .	Many of the new regulation's most-used species have no key in it, and CHOMP returns zero for an absent key without a warning (docs/_reports/2026-09-22-regmc-usage.md §5).	When a consumer switches file, also switch every table it joins that file against. For CHOMP: the usage path, the species table source, and its format constants.
A script that reads process.argv[2] as its input takes --regulation as that input. engine/analyze.js did, as engine/policy.js had.	Under the old regulation with an explicit --regulation , the store path became the string --regulation (docs/_reports/2026-09-22-regmc-usage.md §6).	Fixed abra/regmc 0.23.0. Grep for argv[2] in every script before its first run with --regulation .
engine/status.js --write stamps the DEFAULT regulation's live gate into every division ledger. While data/regulations.json still names the closed regulation active , the new regulation's engine fixes move the closed one's live engine bytes, so its artifacts read <i>measured against a different engine</i> .	On 2026-09-21 a restamp from the main tree would have written the Reg M-B gate as CLOSED into all five ledgers, and would have raised the mtime_only provenance ratchet, on a day the published 7.0.0 record was unchanged on its own frozen release. It was reverted, not committed.	Either flip active (step 8) before the first restamp after the rebuild starts, or make status.js report per regulation. Until then, do not commit a ledger restamp. A closed line's published record is its frozen release, not the live tree.
A key rule that turns every punctuation run into a hyphen splits a base name as if it had a forme. The new table keyed Sirfetch'd as sirfetch-d , and the engine reads the segment before the first hyphen as the base species. The new checkout also spells the name with U+2019, which the one normaliser did not fold.	Five pinned games parted on their very first `	switch ` line, which hid their real causes (four w crit). The old table never had the shape, b punctuated species were illegal there or k hand.

what bit	how it showed	next time
A derivation that hard-codes one value for a whole handler class is right only for the members the old regulation had. The item crit tag gave every <code>onModifyCritRatio</code> item one stage, which was Scope Lens and nothing else in the old format; the returning Leek is two stages and species-locked.	Four pinned games parted on a crit the authority rolled with certainty and this engine rolled at 1/2, hidden until the species name stopped parting the stream first.	When an item class comes back from <code>Past</code>, re-read every member's handler through the tag derivation and diff the regenerated tag file structurally; a constant in a derivation is a claim about the members it has seen. A staged probe of a RATE needs the real-dice (<code>middle</code>) arm; under the bottom arm every crit lands. <code>docs/_reports/2026-09-22-regmc-engine-2.md §2</code>.
A tag whose members had no legal carrier in the old regulation was derived but never consumed, and never checked. <code>allyBasePowerBoost</code> sat in the code's expected-empty list; its parse read <code>chainModify(1.5)</code> as <code>1</code> and no engine code read the tag.	Two pinned games parted on a Steel hit from Perrserker (Steely Spirit): the authority's damage was larger, with nothing in the engine's counters to say a boost had been skipped.	For every tag the old regulation declared expected-empty, check its members' carriers in the new format; if one is legal, print the regenerated params and grep the engine for a consumer before the first differential. <code>docs/_reports/2026-09-22-regmc-engine-2.md §3</code>.
Census rows hard-code the closed regulation's builds. Rows of <code>tests/test-mechanics.js</code> typed a Reg M-B Speed, damage number, free pick, key spelling or authority line into their pass condition.	They read MISSING under the new regulation against an engine that was right; ENGINE had to prove each one a staging gap by hand (<code>docs/_reports/2026-09-22-regmc-engine.md §8</code>).	Fixed 0.40.0: such a row reads its number from the build or from the authority on the run (<code>tests/census_authority.js</code>), or searches for the first candidate whose CONTROL arm stages the mechanic, starting with the historical fixture so the old census stays byte-identical. A new row must never type a number the build or the authority can answer (<code>docs/_reports/2026-09-22-regmc-instruments.md §3</code>).
A table value can mean two things. A census row called any move with <code>bp: 0</code> in the species table a status move, and weight-based attacks carry <code>bp: 0</code>.	Under the new regulation the Taunt row's fixture search picked a body whose "status" click was Low Kick.	Fixed 0.40.0: read the move's category from the <code>statusCategory</code> tag, never infer it from a table field.
Driver state that crosses games. The differential's mega-slot parity flipped on every mega of the whole run and was outside <code>driverSnap</code>.	The same game megaed a different body depending on how many megas ran before it; ENGINE chased an "Aura Guard card" that was this (<code>docs/_reports/2026-09-22-regmc-engine.md §2</code>).	Fixed 0.40.0 for the new regulation: the choice is a per-game address; <code>tests/test-driver-per-game.js</code> plays one game alone, after itself and after another game and requires identical streams. The closed regulation keeps the parity, because its lattices ask the rule (§1 of the instruments report). Grep for module-level <code>let</code> in any driver before a new regulation's first run.
The harness mirror has no answer for a request kind the new regulation adds. Revival Blessing raises a switch request that only a fainted body answers, and the forced-switch mirror looks only for live ones.	Every game that reached a revive THREW, and the refused-choice counter was the only trace.	Fixed 0.40.0: <code>mirrorRevival</code> answers from Showdown's own <code>reviving</code> flag, so the revive is compared on the board (<code>tests/test-revive-mirror.js</code>). For each move new to a regulation, check whether it raises a request the harness has never answered.
A changed value on an entity legal in both regulations is invisible to every added / removed list. Two moves had their PP cut.	Nothing failed: the engine reads PP from the regulation's tag file and happened to be right, but nothing would have said so if it had not been.	Fixed 0.40.0 as a check: <code>tests/probe_regmc_changed_pp.js</code> derives every move legal in both whose <code>pp</code> differs, and runs each out of PP in a real game against the authority. Derive "changed in both" for every entity class, and stage each change.
The roster read the closed regulation's tag file and the old text layout. <code>REL.read('data/tags.json')</code> serves the owner's copy out of every release, and the new checkout does not attach <code>desc / shortDesc</code> to an entity and writes a multiplier as "x".	The first Reg M-C items stage read a large share of its items COULD-NOT-STAGE for no mechanical reason; after the fixes only items new to the format remain (instruments report §4).	Fixed 0.40.0 in <code>tests/roster.js</code>. After a checkout change, print one entity's <code>desc</code> before trusting any rule that reads prose.
A fixture's legality was checked and thrown away. <code>buildPair</code> judged every roster set against the TeamValidator and printed a count on exit; the artifact carried nothing.	The new regulation's roster built sets it refuses and nothing counted them against the gate.	Fixed 0.40.0: <code>fixture_legality</code> travels with the roster counts and <code>engine/quarantine.js</code> fails a stage on a NOT-baselined refusal.
The same item can have different handlers in the two checkouts, and the engine must read which from the tag. White Herb's <code>onAnyAfterMove</code> QUEUES the restore in Reg M-B's checkout (<code>insertChoice</code>, a queued event) and RUNS it inside <code>useMove</code> in Reg M-C's, so only the new regulation spends it on the move that ends the battle.	A fix written for the new regulation alone moved the closed one: the Reg M-B lattice went from 0 to 4 board-material games, every one a White Herb the old authority kept.	Before fixing a new-regulation card, print the entity's handlers in BOTH checkouts; when they differ, derive the difference into a tag field that is written only for the new shape (so the old tag file derives byte-identically) and gate the engine on it. Run the old lattice on every commit: it is the control that caught this. <code>docs/_reports/2026-09-22-regmc-engine-2.md §4</code>.
A road "counted, not modelled" in the old regulation can be dormant there by construction. Revival Blessing's revive road was a <code>MEDFAILS.reviveUnmodelled</code> counter that no Reg M-B game could ever raise (the move and its one learner are <code>Past</code> in Reg M-B), so the closed line's zero said nothing about it.	Six of the thirteen pinned Reg M-C board-material games at 0.40.0 were that one counted road.	On a rotation, list every <code>MEDFAILS</code> counter whose entities are illegal in the old regulation and legal in the new one: each is an unmodelled mechanic the old gate never exercised. Fixed 0.41.0 (<code>reviveFainted</code>).
An ability with no legal carrier in the old regulation is untagged in the new one, and the usage floor hides it. Liquid Ooze is legal in both formats but carried only in Reg M-C, so Reg M-B's tag file never had its row, and <code>engine/tag_dex.js</code> flags an	A drain into it healed the drainer, and the only sign was one board-material game in the pinned differential.	On a rotation, list every ability, item and move that is <code>untagged</code> in the new tag file AND has a legal carrier there, regardless of usage, and read each handler. Fixed 0.42.0 (<code>reversesHeal</code>).

what bit	how it showed	next time
untagged entity only above 0.5% usage (<code>UNTAGGED_FLOOR</code>): 4 Reg M-C sheet uses printed nothing.		
The new pool deals pairs the old lattices never did, and a defect it finds can belong to the closed line too. Berserk ignored Sheer Force's skip of <code>AfterMoveSecondary</code> in BOTH regulations: both abilities are Reg M-B legal and the two checkouts share the gate, but no Reg M-B lattice dealt Mega Camerupt's Earth Power into a Drampa.	A Reg M-C card that is really a shared-engine defect; the fix changes Reg M-B behaviour while every Reg M-B instrument reads unmoved.	Before fixing a new-regulation card, ask whether its entities and handlers are legal and identical in the old checkout; if so, say the fix is shared, run the old lattice, and name the old held-out draw as owed. Fixed 0.44.0 (<code>sheerForceSkipsAfterMove</code>).
A tag keyed on one handler misses a member with the same effect in another handler. <code>failsWithoutTerrain</code> reads Steel Roller's <code>onTry + onHit</code> ; Ice Spinner clears the terrain from <code>onAfterHit</code> and has no <code>onTry</code> , so it matched nothing and the engine never removed a terrain for it -- in Reg M-B too, where it has 299 sheet uses (<code>data/tags.json:moves.icespinner.uses</code> ; <code>~~301~~</code> before the 0.111.2 regeneration re-weighted usage from the stores) and every gate lattice read zero (no lattice dealt it into a standing terrain before the terrain ran out).	A Reg M-C card that is a shared-engine gap, invisible to every Reg M-B instrument.	When a card names an EFFECT (<code>clearTerrain</code> , <code>setTerrain</code> , <code>addSideCondition</code>), grep that call across every handler of every legal entity in both checkouts, not the handler the existing tag reads. Fixed 0.45.0 (<code>clearsTerrainAfterHit</code>).
A derivation that refused a condition "for now" was safe only while the old regulation had no carrier. <code>condStatMult</code> refused Grass Pelt's terrain-gated Defence ("a real gap left open rather than guessed at"); in Reg M-B no legal species has the ability, so the refusal cost nothing. Reg M-C legalised Gogoat and the refusal became a missing x1.5.	A Reg M-C card on a damage roll (Dire Claw into Gogoat) whose cause was a tag documented as deliberately absent.	On a rotation, grep <code>engine/tag_dex.js</code> for refusals (<code>return null</code> ; <code>/* a condition this derivation cannot name</code> , "left open", "REFUSED") and re-check each refused entity's carriers in the new regulation. Fixed 0.46.0 (<code>condStatMult.when: 'terrain'</code>).
A rule fixed at one door has to reach every door of the same shape, and the new regulation can open doors the old one kept shut. The 2026-09-20 fix that addresses a pivot entrant's dice as a new action (<code>pivotFrom</code>) covered U-turn; Emergency Exit and Eject Button raise the same <code>switchFlag</code> , but neither had a Reg M-B carrier (Eject Button <code>Past</code> , Emergency Exit no species), so their entrants kept the stale address.	Reg M-C's first Emergency Exit Trace drew a different shared die than the authority and copied the other foe's ability.	When a fix names a mechanism (<code>switchFlag</code> , <code>forceSwitchFlag</code> , a queue re-entry), list every entity that reaches that mechanism in the NEW checkout and route it through the same helper. Fixed 0.48.0 (<code>midAddrOwnAction</code>).
A gap a code comment names as "still unpaid" is dormant only while the old pool rarely deals it. The max-HP recoil block said in so many words that Steel Beam into a Protect was unpaid; in Reg M-B no gate lattice dealt it, and Reg M-C's Lucario-Mega-Z made it a turn-1 opener three times in one lattice.	A turn-1 HP gap on the attacker, the same number every time.	On a rotation, grep the engine for the gaps it names in its own comments ("still UNPAID", "not modelled", "named rather than") and re-check each against the new pool's usage before the first lattice. Fixed 0.49.0 (<code>_failRecoilOnShield</code>).
A second copy of an ordering rule stays wrong wherever the first copy was corrected. Magician sorted its targets with its own <code>effSpeed</code> comparator after <code>sdEachEventOrder</code> had been taught the authority's cached, Trick-Room-negated <code>speedSort</code> ; the Reg M-B lattices never dealt a Magician spread hit under Trick Room.	A Reg M-C item theft from the wrong foe, only under Trick Room.	When a card is an ORDER (who is asked first), grep every <code>speedSort / .sort(</code> over bodies in the engine and route each through the one emulation. Fixed 0.50.0 (<code>sdSpeedSortEntries</code>).
A question asked of the wrong body is right whenever the two bodies agree, and a new pool changes how often they do. The Psychic Terrain gate asked the aimed body rather than the Follow Me user it was drawn to; the two differ only when the aimed body is airborne and the drawer grounded, which Reg M-C's Talonflame + Indee Dee cores deal on turn 1.	A priority move that lands here and is refused by the terrain on the authority.	For every per-target refusal (<code>onTryHit</code> , <code>onTryImmunity</code>), check that the engine asks the post-redirect target; grep the engine's comments for "post-redirect" and "named rather than folded in". Fixed 0.51.0.
A refusal modelled at the late site passes a case the early site catches. The engine refused a gone body's action at <code>runAction (isActive)</code> ; the authority also cancels it at <code>switchIn (cancelAction)</code> , which matters only when the body comes BACK the same turn -- two forced exits in one turn, which needs Reg M-C's Eject Button and Emergency Exit together.	A body acting on a turn it was ejected.	When a new regulation legalises a second way to leave or re-enter the field, re-read <code>switchIn / cancelAction / runAction</code> and ask what a body that leaves AND returns in one turn keeps. Fixed 0.52.0 (<code>TURN_EPOCH</code>).
The new regulation's checkout may override a function the old notes cite from mainline. 0.45.0 cited <code>sim/battle-actions.ts</code> 's <code>if (moveData.onAfterHit && pokemon.hp)</code> ; the Champions mod ships its own <code>spreadMoveHit</code> without the guard, so a user knocked out by a contact toll still runs its move's <code>onAfterHit</code> .	A terrain (or a hazard) that should have ended, left standing after the attacker fainted to Rocky Helmet.	Before citing a <code>sim/</code> line as the rule, grep <code>data/mods/champions/scripts.ts</code> in BOTH checkouts for the same method name (<code>spreadMoveHit</code> , <code>hitStepMoveHitLoop</code> , <code>modifyDamage</code> , <code>getActionSpeed</code> ...). Fixed 0.53.0 for Ice Spinner; the hazard families are named.
A tag that admits several target classes is read as the one class the old regulation used. <code>healsAlly</code> admits every friendly class; in Reg M-B its only pair-sized member was Life Dew (<code>allies</code>), so the engine read the tag's presence as "heal both". Champions M-C retargets Milk Drink to <code>adjacentAllyOrSelf</code> and it joined the tag.	A heal that restores the partner as well as the user.	When a mod row changes a move's <code>target</code> , list every tag the move carries and grep the engine for readers that key on the tag's presence rather than on <code>targetClass</code> . Fixed 0.54.0 (<code>healParam</code>).
A door that is the third caller of an entry routine can miss the routine's tail. <code>megaEvolveNow</code> ran <code>applyEntryEffects</code> (the mega's weather) without the <code>syncFieldTypes</code> the switch road ends in; Reg M-B's pool never put a Castform beside a weather mega, Reg M-C's did (Frolass-Mega).	A Forecast / Mimicry body on the wrong forme after a mega's weather or terrain.	When a new regulation adds a weather- or terrain-setting mega, grep every <code>applyEntryEffects(</code> caller and check each ends in the same field sync. Fixed 0.55.0.
A refusal written early in the engine skips every step the authority runs between it and the real refusal. The Psychic Terrain gate sits at the top of the attack path but refuses at <code>TryHit</code> ; Protean's <code>PrepareHit</code> conversion lies between. Reg M-B's pinned pool never dealt a Protean priority move into Psychic Terrain; Reg M-C's (Greninja + Indee Dee) did.	A Protean / Libero body keeping its old types after a terrain refusal.	For each early <code>continue</code> in the attack path, list the authority steps between the engine's position and the refusal's real step (<code>Try</code> , <code>PrepareHit</code> , the hit steps) and check none writes state. Fixed 0.56.0.
An engine that does a faint's cleanup at the HP-zero moment is wrong for every handler the authority runs between that moment and faintMessages. <code>faintHousekeeping</code> reverts a transformation off <code>noteFaint</code> ; the <code>DamagingHit</code> reactors run after it here and before <code>clearVolatile</code> there. Reg M-B's pool never KO'd	A toll, a buff or a drop owed by the copied ability, missing on the hit that KO'd the copy.	For every state <code>noteFaint</code> rewrites, ask which handlers still read it before <code>faintMessages</code> (<code>DamagingHit</code> , <code>AfterHit</code> , the self drops) and route them through the worn value (<code>_abAtFaint</code> , <code>dhAbilityOf</code>). Fixed 0.58.0 for the <code>DamagingHit</code> reactors.

what bit	how it showed	next time
a transformed Rough Skin copy with contact; Reg M-C's (Ditto + Garchomp) did.		
A new regulation's move can create a body that is in <code>side.active</code> and not <code>isActive</code>. Revival Blessing (M-C only) revives into an empty active slot and defers the instaswitch behind the residual; every <code>this.heal / damage / boost</code> refuses the body meanwhile. Reg M-B had no road to that state.	A residual heal or chip on a revived body before it walks in.	When a new move revives, swaps or drags bodies, list what it leaves in <code>side.active</code> without <code>switchIn</code> (the <code>isActive</code> setter) and check each engine walk over the actives skips it. Fixed 0.59.o (<code>_revivePending</code>).
A new regulation's entities fall to the roster's residue rule, which stages nothing. The shape rules were written for the old regulation's entities; fifteen Reg M-C items and abilities (Air Balloon, Rocky Helmet, the seeds, Run Away's Champions escape...) matched no rule and read COULD-NOT-STAGE "THE STAGING IS INERT" on every Reg M-C roster since the regulation opened.	A block of COULD-NOT-STAGE rows whose rule is <code>item/held-and-nothing-more</code> or <code>ability/generic</code> .	After the first roster of a new regulation, list every row by <code>rule</code> and treat each residue row as owed fixture work: write the shape rule that creates what its handler waits for. Fixed 0.64.o.
A mod that changes a move's target class can outrun the scripted driver's encoder. Champions M-C makes Milk Drink <code>adjacentAllyOrSelf</code> ; <code>scripted()</code> resolved that class to the user's own slot whatever the script asked, so no staged scenario could aim it at the partner.	A heal that lands on its full-HP user, read as an inert row.	When a mod row changes <code>target</code> , check <code>scripted()</code> in <code>engine/game_differential.js</code> can express every aim the new class admits. Fixed 0.64.o (<code>{ ally: true }</code>).
A tag's own "no ability does this" note is a fact about the regulation it was written in. <code>escapesTrap</code> was item-only because "exactly ONE item in Reg M-B declares <code>onTrapPokemon</code> , and NO ability does"; the Champions M-C mod gives Run Away that exact handler.	A trapped body that the authority lets switch and this engine holds.	On a rotation, <code>grep engine/tag_dex.js</code> for membership claims ("NO ability", "exactly ONE", "matches nothing") and re-ask each over the new dex. Fixed 0.66.o.
A row below the usage shelf is shelved on a count, not measured clean, and a new regulation's store is thin. Bounce (2 clicks) and Jaw Lock (o) sat on the shelf over a real board DIFFER - one a shared-rule defect (the charge-turn <code>PrepareHit</code>) that reaches Reg M-B's Greninja too.	A BELOW-USAGE-SHELF row whose <code>underlying_verdict</code> is FIRED-AND-BOARDS-DIFFER.	Read every shelf row's underlying verdict on a new regulation's first rosters and diagnose each DIFFER against the authority; a thin store is a sampling fact, not a correctness one. Fixed 0.67.o (Bounce) and 0.68.o (Jaw Lock).
A probe lead that read FALSE in the old regulation ("nothing legal reaches it") has no arm to run when the new regulation legalises the entity. <code>probe_protean_contrary</code> 's <code>itemboost</code> lead derived Reg M-B's empty stat-raising item list and staged no arm. Reg M-C legalises the terrain seeds, so the derivation found them and the lead read NOT STAGED from 0.62.o on.	A probe that is green in the old regulation and red on a <code>NOT STAGED</code> lead in the new one.	On a rotation, run every probe with an authority-only or FALSE lead under the new regulation, and write arms for every entity its derivation newly finds. Fixed 0.69.1.
Two checkouts can attach move text differently, and a wrapper written for one throws on the other. The Reg M-C checkout builds entities with no text, so the roster wrapped them in a Proxy that reads the text table. Reg M-B's checkout carries 16 Hidden Power typings whose text is a FROZEN own <code>' '</code> , and a Proxy may not report another value for such a property. Every shape rule that walks <code>moves.all()</code> threw, and 7 abilities went COULD-NOT-STAGE on the ruler.	<code>'get'</code> on proxy: property <code>'shortDesc'</code> is a read-only and non-configurable data property inside a COULD-NOT-STAGE reason.	Whenever a view wraps dex entities, test it against every live checkout, not only the new one (<code>tests/test-roster-text-view.js</code>). Fixed 0.78.o.
An instrument change made for the new regulation can change what the OLD regulation's gate reads, and the old gate only shows it when it is re-measured. <code>tests/roster.js</code> added <code>fixture_legality</code> (0.40.o), and <code>engine/quarantine.js</code> counts a refused set against the clause on the understanding that Reg M-B artifacts carry none. The first Reg M-B roster re-run carried the block (17/77/47 refused sets, <code>data/roster.items.json:fixture_legality.not_baselined</code> and its sibling stages) and turned three PASS clauses into FAIL. The same pass's text proxy threw on Reg M-B's frozen <code>' '</code> Hidden Power text and knocked 7 abilities to COULD-NOT-STAGE.	Old-regulation clauses turning red on a re-measure while every DIFFER count stays o.	Re-run the OLD regulation's roster in the same pass as any change to a shared instrument, and diff its bucket counts against the previous artifact. <code>docs/_reports/2026-09-23-gates-on-finished-engine.md</code> .
The two regulations' checkouts are different Showdown versions, so a mainline handler can differ between them even where the Champions mod overrides nothing. Stone Axe's and Ceaseless Edge's <code>onAfterHit</code> asks <code>source.hp</code> in the Reg M-B checkout and not in the Reg M-C one. 0.60.o read only one checkout and changed both regulations.	A shared-rule fix whose probe was run under one regulation only; the other regulation's held-out draw parts.	For a fix marked BOTH REGULATIONS, read the handler in both checkouts, derive the difference into a tag param, and run the probe under both. Fixed 0.71.o.
A Champions mod override in one regulation's checkout can be absent from the next one, and then mainline is the authority. Reg M-B's mod overrides Disguise to hold the neutral for a whole volley (<code>effectState.neutral</code>). Reg M-C's mod has no disguise entry, so the busted forme takes its real matchup from arrival 2 on. The engine had one rule for both, and it was wrong a different way in each regulation (battle in M-B, price in M-C).	A multi-hit into a Mimikyu parts on the board in one regulation and on the damage differential in the other.	On a rotation, diff each checkout's <code>data/mods/champions/*.ts</code> entry list against the last one, and re-derive every tag param read off a handler the diff touches. Fixed 0.73.o.
A roster shape rule stages ONE handler of an ability, and a new regulation can make the unstaged handler matter. Guard Dog's roster rule is the <code>onDragOut</code> phaze refusal, and Rattled's is the <code>onDamagingHit</code> type trigger. Neither stages the Intimidate half (<code>onTryBoost / onAfterBoost</code>), so both read FIRED-AND-BOARDS-MATCH while the Intimidate reaction was missing from the engine. Both abilities are new in the M-C carrier pool.	The roster is clean and <code>all_mechanics_fire</code> reads STATE on the same ability.	For each newly legal ability, list every <code>on*</code> handler it declares and name the instrument that stages each one. A handler that no instrument stages is not tested. Fixed 0.74.o (Guard Dog) / 0.75.o (Rattled).
A protocol string the old regulation's checkout spelled one way can be spelled another way in the new one, with no Champions override and no rule change. The Intimidate refusers write <code>'Attack'</code> in the Reg M-B checkout and <code>'atk'</code> in the Reg M-C one. The engine's display table held the Reg M-B spelling, so every Reg M-C refusal line parted.	Narration-only partings that cluster on one line shape across many abilities.	Diff the <code>this.add(...)</code> literals of every handler the engine reproduces between the two checkouts, and read each literal into a tag param instead of a table. Fixed 0.76.o.
A control built from a NAMED class (the "quiet" abilities) can have no legal form, even though the property it needs does. The roster's Skill Swap lender had to hold a quiet ability. No	<code><Species></code> can't learn <code><move></code> . on a control arm or a pool	Ask what the control must BE (board-inert), and derive every class that has that property (<code>announces-only</code> ; a Ground immunity on

what bit	how it showed	next time	
legal quiet holder learns Skill Swap in either regulation, so the control arm of every swap-controlled ability row was a set the TeamValidator refuses. Heal Bell, Roost and Transform had the same shape on the move stage: the pool held no legal learner.	body, with a staged row above it that reads MATCH.	a board proven Ground-free; a one-status ability on a board that writes no such status). Re-derive the pool per regulation. Never keep the refused set, and never baseline it. Fixed 0.79.0 (docs/_reports/2026-09-23-roster-last-refusals.md).	
The scope authority re-admits a Future -flagged entity, and every instrument that keeps its own strict filter drops it without saying so. engine/legal_scope.js admits Reg M-C's Aura Guard through the TeamValidator. The staging planner's universe and all_mechanics_fire's population filtered !isNonstandard themselves, so Aura Guard had no fixture and NO ROW. The gate read that as unproven.	A mechanic that is in scope, with a tag and a probe, and with no row in one instrument.	Take a population from legal_scope , never from a local filter, and print the re-admissions. A CLI that defaults to data/tags.json is reading Reg M-B's catalogue under --regulation regmc , so pass the file. Fixed 0.77.0.	
A hand-copied snapshot of an authority flag does not grow when the regulation adds a member. SUBPASS was the bypasssub flag copied into a 51-id literal. Reg M-C made Overdrive legal and the literal did not have it, so every Substitute blocked it. The conformance clause saw it; nothing else did.	The damage differential exits 1 on a conformance block with 0 damage disagreements.	Grep the engine for literal id sets that mirror a move, ability or item flag, and derive each into a tag before the rotation. Fixed 0.80.0 (bypassesSubstitute).	
A move that is Past in the old regulation can have NO engine implementation at all, and nothing reports it until something stages it. Court Change reached the terminal pass (a no-op turn); only the planner's constructed fixture (a side condition raised first) showed the board STATE divergence.	all_mechanics_fire reads resolved-on-the-authority-only for a newly legal move.	For each newly legal move, check that playerAction returns a kind other than the terminal pass. Fixed 0.82.0 (swapsSideConditions).	
A probe of a move new to the next regulation finds no learner in the old one and exits 2, which reads as COULD-NOT-STAGE. probe_court_change under Reg M-B: the move is Past there, so no fixture can exist, yet exit 2 was booked against the gate.	A shared-rule probe exiting 2 "no legal learner" in the regulation that does not carry the entity.	Ask engine/legal_scope.js first; when the entity is out of scope, assert it (dex flag, zero legal carriers, validator existence refusal with a cleared control) and exit 0 as NOT-IN-REGULATION. Exit 2 is never a verdict about a mechanic. Fixed 0.87.1.	
A probe written under one regulation hard-codes that regulation's checkout, release or mod block. Pass 9 found three: a default SHOWDOWN_PATH to the Reg M-B checkout, a default --release pinned to a Reg M-B release, and an assertion that the Champions mod carry a disguise block. Each read red or CANNOT ANSWER under Reg M-C with a correct engine.	A probe reds or throws under the new regulation on an assertion about the checkout, not the mechanic.	Grep tests/ for literal checkout paths and release ids; resolve through engine/showdown_path.js and the newest release for the selected regulation. Fixed 0.82.1.	
A REGISTER instrument can hardcode the old regulation too, and then an open row reads CANNOT-ANSWER in the new regulation's defect clause. The 0.82.1 grep covered probes the gate stages, not the ROADMAP markers engine/register_reality.js runs. #310's probe forced the Reg M-B checkout and pinned a Reg M-B release; #442's forced the checkout and read the census as data/mechanics-census.json , so it found the old regulation's copy. 53 more files in tests/ and engine/ still default SHOWDOWN_PATH to the Reg M-B checkout, 17 of them named in a ROADMAP VERIFIED BY: cell.	"no open known engine defect" CANNOT-ANSWER under the new regulation, with the refusal naming a checkout, a release or a timestamp rather than the mechanic.	Grep every VERIFIED BY: script for literal checkout paths, release ids and unsuffixed artifact names; resolve through engine/showdown_path.js , the selected regulation's current release and artifactFor . Fixed 0.100.3 for #310 and #442.	
A once-per-body latch is correct until a regulation adds a way to come back. The trace wrote `	faint	` once per body and never reset it, which was right while nothing revived; Reg M-C's Revival Blessing lets a body die twice, and the second death was silent.	Whole-game narration rows reading ``
A handler's inline exclusion list is part of the mechanic, and a tag that reads only the headline drops it. Every -ate onModifyType opens with const noModifyType = [...] (Weather Ball, Terrain Pulse, ...); convertsMoveType carried the type and the multiplier and not the list, in both regulations, so a Pixilate or Refrigerate Weather Ball converted and took x1.2.	The Reg M-C staged harness reported "Pixilate parts boards" and "Refrigerate unproven" (its control clicked a different move); neither the damage differential nor the pinned pool clicks an -ate Weather Ball, so nothing else saw it.	When a tag reads a handler, grep the handler for a literal array or an includes(move.id) and carry it as a param. A probe that stages the ability must also stage one move on the list. Fixed 0.106.0 (convertsMoveType.except).	
The staged harness's move picker reads the same handlers, and it dropped the same list. fixture_preflight derived an -ate need as type=Normal alone, and stage_planner.triggersOf copied only its kind and values. So the planner staged Pixilate and Refrigerate with Weather Ball, the one Normal move the handler skips, in both regulations.	On the fixed engine Pixilate's planner arm read DID-NOT-FIRE, and the row printed FIRED only because the legacy fallback rescued it. Nothing flagged it.	When a derived need gains a field, check that every copier of the need carries it. tests/probe_ate_picker.js plans every exclusion-bearing row and prints every handler that excludes a move id, so a new regulation's member arrives named. Fixed 0.107.0 (except on the need).	
A field condition that refuses a move has a CALLED half as well as a CHOSEN half, and a move flag no artifact carries is invisible to both. Gravity refuses a gravity-flagged move in onBeforeMove (chosen) and in onModifyMove (called through useMoveInner , which skips BeforeMove). No tag carried flags.gravity , so neither half existed in either regulation.	A scripted Sleep Talk and Copycat under Gravity played High Jump Kick, and a chosen Bounce started its charge. Neither the pinned pool nor the damage differential staged it, and nothing flagged it.	On a rotation, list every legal caller of actions.useMove((the legal set changes between regulations) and every condition whose onModifyMove writes a cant . Check that the engine asks it for a _copied action. Fixed 0.108.0 (refusedByPseudoWeather).	
The new checkout's sim/ can drop a guard the engine copied, with no Champions override and no data-file change. Reg M-C's checkout carries upstream efe4948 , which removed getMoveTargets's isCharging guard; the engine had copied the guard from the Reg M-B checkout, so a charge turn drew no redirect in either regulation.	A Lightning Rod / Storm Drain - activate missing above a real - prepare under the new regulation only, and a Pressure PP leaf off by one on a charge turn.	On a rotation, list the new checkout's commits to sim/ between the two pinned commits and re-read every sim line the engine cites; derive each behavioural difference into a tag param read off the compiled method. Fixed 0.92.0 (chargeTurn.drawnWhileCharging).	

what bit	how it showed	next time	
A mod can delete a declared volatileStatus and add the volatile inside onHit , which moves the volatile BELOW the move's own effects. Reg M-B's Curse declares volatileStatus: 'curse' (added by runMoveEffects above onHit); the Reg M-C mod sets it undefined and calls directDamage and then addVolatile inside onHit , so the user's -damage now precedes the -start . No rule changed; only where the add happens.	A two-line swap on one move's -start and -damage , narration only, in the new regulation alone.	On a rotation, list every mod entry that sets volatileStatus: undefined (or drops volatileStatus / sideCondition) and re-read its onHit for the order of the add against the move's other lines; derive it into a tag param. Fixed 0.90.0 (typeSplitMove.costBeforeVolatile).	
A Champions mod override can disappear between regulations, and every engine road built to mirror it keeps running. Reg M-B's mod QUEUES White Herb's after-move restore (data/mods/champions/items.ts:1023-1037); Reg M-C's mod has no whiteherb entry, so mainline's immediate restore stands. The engine's queued road (pivotHerbSweep , built for Reg M-B) went on firing under Reg M-C, and the herb came out below a Parting Shot's `	switch	` instead of above it.	One whole-game narration row (Parting Shot's a White Herb holder); tests/probe_narration_b_line_or herb arm, written against the Reg M-B read FIXTURE FAILED under Reg M-C.
Renaming a division is not a file move. SEARCH became SOLVER on 2026-09-24. build/build_pdfs.js derives the ledger set from .claude/agents/*.md , so it followed the agent file, but three places typed the ledger name: engine/status.js SECTIONS (its key IS the ledger's file name), tests/test-roadmap-register.js LEDGERS , and a key in tests/test-docs-quarantine.js ; and data/docs-currency-baseline.json lists the unversioned ledger by path.	Nothing would have failed loudly: status.js --write prints skip SEARCH.md (not present) and leaves the renamed ledger unstamped, and the register test stops reading one ledger (docs-refresh-b, 2026-09-24).	When a division or ledger is renamed, grep engine/ , tests/ , build/ and data/*.json for the old ledger path and the old upper-case name, and change them in the same commit. Leave dated references in code comments; the renamed ledger says where they resolve.	
An engine fix that repeats a line can silently disarm a roster red demonstration. 0.120.0's restoreStatsOwned copied restoreStatsUpdate's _rs read; the item/restores-lowered-stats plant in tests/roster.js then matched twice, refused to apply, and the items stage exited 1 with 166/166 green, which the gate reads as a failing roster clause.	Found by the Reg M-C final gate re-read: the roster/items clause read FAIL with o DIFFER and o DID-NOT-FIRE, and reds in data/roster.items-regmc.json said the anchor matched 2 times.	After any engine change, read the roster's N of N apply exactly once line before trusting a stage exit. Re-aim a plant at a line only its own door carries, never at a line a new helper might copy (0.124.0).	

THE THING THAT WILL GO WRONG ANYWAY

Every failure this rotation found had the same shape as every failure this repository has ever had: a **capability absent, with everything reporting success**. The same-bytes shortcut reported *+o moves*. The strict legality filter reported full coverage over an ability it had deleted. A silent config fallback would report a clean run about the wrong regulation.

None of those crashes. So the standing question at every step is not *did it run* but **could this have told me if it were wrong** — and where the answer is no, print the count, print the re-admitted list, print the fallback. A zero that nothing could have made non-zero is not a measurement.